

EC801 Logic Synthesis Assignment 2

Team Members

1. Rohit L (231EC248)
 2. Pranav Jois (231EC237)
 3. Manal Ahmed (231EC135)
-

Introduction

This assignment implements the construction and display of a Reduced Ordered Binary Decision Diagram (ROBDD) for a Boolean function given in Sum of Products (SoP) form.

The program accepts a Boolean expression from an input text file and converts the expression into an internal representation. The variables are arranged according to a variable ordering generated by the program. The Boolean function is then recursively decomposed using cofactors with respect to the variables in the selected order.

During ROBDD construction, redundant nodes are removed and equivalent nodes are shared. The final ROBDD is represented as a directed graph containing variable nodes and two terminal nodes, 0 and 1.

The program also displays the ROBDD in both Depth First Search (DFS) and Breadth First Search (BFS) formats. In addition, the ROBDD is graphically generated and saved as an image.

The implementation is done in one Python file:

231EC237_231EC248_231EC135_Code2.py

These are the important functions/files used to create the ROBDD:

- bool_func – Boolean expression processing and ROBDD construction
- tree_node – Tree node and DFS/BFS implementation
- draw_robdd – Graphical representation of the ROBDD
- test.txt – Sample Boolean expressions given by the user (to be created by user)

The code can be accessed here:

<https://github.com/RohitLalu/EC801-Logic-Synthesis-Coursework/tree/main/q2>

Naming Convention (same as Assignment 1)

- Normal Boolean variable: lowercase letter

- Complemented Boolean variable: uppercase letter

For example: a represents a , A represents $\bar{a}$

Therefore, $aB + Ab$ represents: $ab + \bar{a}\bar{b}$

Input Format

The Boolean expressions are stored in a text file with one Boolean expression (SOP Form) on each line. Product terms are separated using the + operator.

For example, the test.txt file used for testing contains:

```

main.py X test.txt X
EC801-Logic-Synthesis-Coursework > q2 > test.txt
1  a + Ab
2  aB + Ab
3  ab + Cd + eF
4  ab + Cd + eF + Ag + BH
5  ABC + Abc + bC + aBc
6  ABC + Abc + bC + aBc + ab
7  a+ bc + DE + FGH + Af + IjK + lMN + bn + dhk + eIn
  
```

The program first asks for the name of the input file and then asks for the expression number corresponding to the line that has to be converted into an ROBDD, each line number corresponds to an expression. Then the user enters the variable ordering.

No spaces are required between variables inside a product term.

Variable Ordering (given by user)

The program asks the user to enter the variable ordering. Example: abc implies the ordering is $a < b < c$. Enter the variables without spaces. If all the variables are not entered, error will show up.

The usual approach is to check the frequency of the variables and keep the most frequent one as higher in the order. This is important because the size and structure of an ROBDD depend strongly on the ordering of the variables.

Software Requirements

The program requires: Python 3, NetworkX, Matplotlib. The required Python packages can be installed using:

pip install networkx matplotlib

Steps to Execute the Program:

1. Place the Python file and test.txt file in the same directory.
 2. Open a terminal in that directory.
 3. Run: **python 231EC237_231EC248_231EC135_Code2.py**
 4. Enter the input text file name: For example: **test.txt**
 5. Enter the expression number. For example: 5
 6. Enter the variable ordering: For example: abc
 7. The program constructs the ROBDD.
 8. The program displays the DFS traversal.
 9. The program displays the BFS traversal.
 10. The program generates the graphical ROBDD.
 11. The graphical ROBDD is saved as: **robdd.png** in the same folder.
-

Internal Representation (for consistency)

The Boolean expression is first divided into individual product terms/cubes.

The program rearranges the variables within each product term into a consistent order.

For example, an expression such as: Bac, can be internally rearranged as: aBc

PCN Representation

The Boolean cubes are represented internally using a three-state representation called PCN. For every variable:

- 1 represents the variable in its normal form.
- 2 represents the complemented form of the variable.
- 3 represents that the variable is not present in the product term.

For example, consider the cube: aBc. For variables a, b, and c: a is present → 1
b is complemented → 2 c is present → 1

Therefore, the cube can be represented as: a = 1, b = 2, c = 1

If a variable is absent from a cube, its value is represented by 3.

This representation makes it possible to efficiently determine the cofactors of the Boolean function.

Results:

Screenshots of Execution (initial user inputs)

```
PS C:\Users\ASUS\OneDrive\Documents\NITK\7th Sem\Logic Synthesis\Assignment 2> python 231EC237_231EC248_231EC135_Code2.py
Enter test file name: test.txt
Enter which expression to convert to ROBDD: 5
Enter variable ordering: bca
```

Test Case 1

Input: $a + Ab$

Expression Number: 1

Variable Ordering: a, b

```
PS C:\Users\ASUS\OneDrive\Documents\NITK\7th Sem
Enter test file name: test.txt
Enter which expression to convert to ROBDD: 1
Enter variable ordering: ab
DFS:
id=3 var=a low=2 high=1
id=2 var=b low=0 high=1
id=0 TERMINAL 0
id=1 TERMINAL 1
BFS:
id=3 var=a low=2 high=1
id=2 var=b low=0 high=1
id=1 TERMINAL 1
id=0 TERMINAL 0
ROBDD written to robdd.png
```

The ROBDD contains two non-terminal variable nodes and the two terminal nodes.

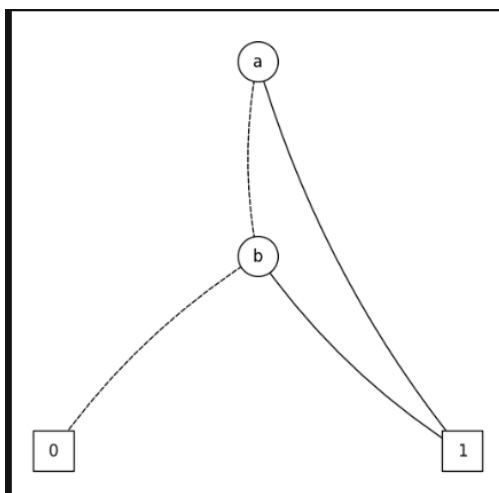


Figure 1: ROBDD generated for Test Case 1

Test Case 2

Input: $aB + Ab$

Expression Number: 2

Variable Ordering: a, b

```
PS C:\Users\ASUS\OneDrive\Documents\NITK\7th S
Enter test file name: test.txt
Enter which expression to convert to ROBDD: 2
Enter variable ordering: ab
DFS:
id=4 var=a low=2 high=3
id=2 var=b low=0 high=1
id=0 TERMINAL 0
id=1 TERMINAL 1
id=3 var=b low=1 high=0
BFS:
id=4 var=a low=2 high=3
id=2 var=b low=0 high=1
id=3 var=b low=1 high=0
id=0 TERMINAL 0
id=1 TERMINAL 1
ROBDD written to robdd.png
```

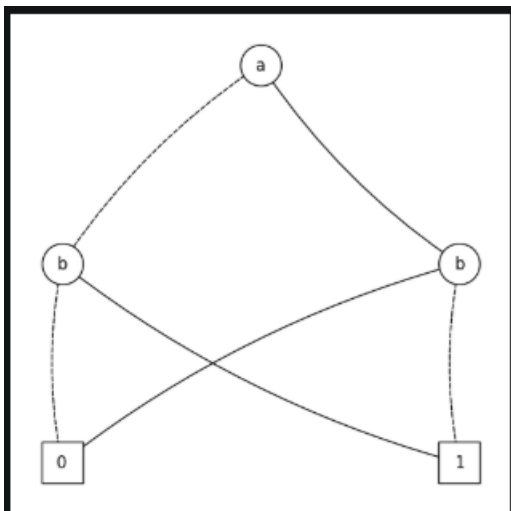


Figure 2: ROBDD generated for Test Case 2

Test Case 3

Input: $ab + Cd + eF$

Expression Number: 3

Number of Variables: 6

Variable Ordering: a, b, c, d, e, f

```
PS C:\Users\ASUS\OneDrive\Documents\NITK\7th S
Enter test file name: test.txt
Enter which expression to convert to ROBDD: 3
Enter variable ordering: abcdef
DFS:
id=7 var=a low=5 high=6
id=5 var=c low=4 high=3
id=4 var=d low=3 high=1
id=3 var=e low=0 high=2
id=0 TERMINAL 0
id=2 var=f low=1 high=0
id=1 TERMINAL 1
id=6 var=b low=5 high=1
BFS:
id=7 var=a low=5 high=6
id=5 var=c low=4 high=3
id=6 var=b low=5 high=1
id=4 var=d low=3 high=1
id=3 var=e low=0 high=2
id=1 TERMINAL 1
id=0 TERMINAL 0
id=2 var=f low=1 high=0
ROBDD written to robdd.png
```

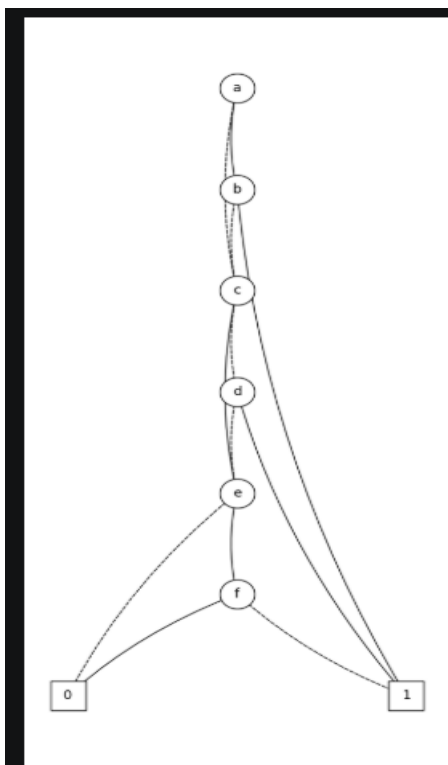


Figure 3: ROBDD generated for Test Case 3

Test Case 4

Input: $ab + Cd + eF + Ag + BH$

Expression Number: 4

Number of Variables: 8

Variable Ordering: a, b, c, d, e, f, g, h

Beginning of DFS Output

id=19 var=a low=13 high=18

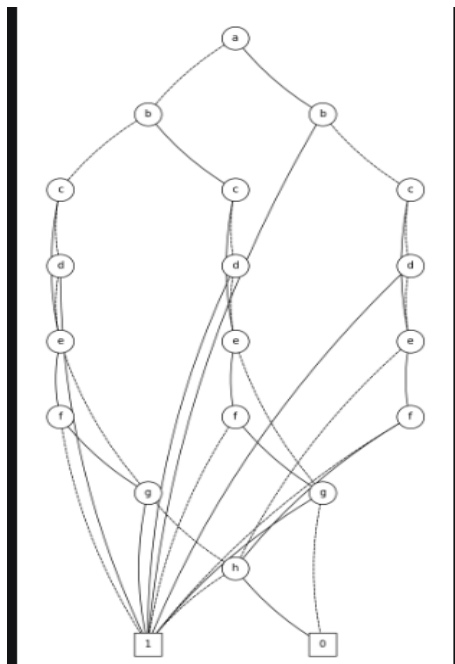


Figure 4: ROBDD generated for Test Case 4

Test Case 5

Input: $ABC + Abc + bC + aBc$

Expression Number: 5

Number of Variables: 3

Variable Ordering: b, c, a

```

PS C:\Users\ASUS\OneDrive\Documents\NITK\7th S
Enter test file name: test.txt
Enter which expression to convert to ROBDD: 5
Enter variable ordering: bca
DFS:
id=6 var=b low=4 high=5
id=4 var=c low=2 high=3
id=2 var=a low=1 high=0
id=1 TERMINAL 1
id=0 TERMINAL 0
id=3 var=a low=0 high=1
id=5 var=c low=1 high=2
BFS:
id=6 var=b low=4 high=5
id=4 var=c low=2 high=3
id=5 var=c low=1 high=2
id=2 var=a low=1 high=0
id=3 var=a low=0 high=1
id=1 TERMINAL 1
id=0 TERMINAL 0
ROBDD written to robdd.png

```

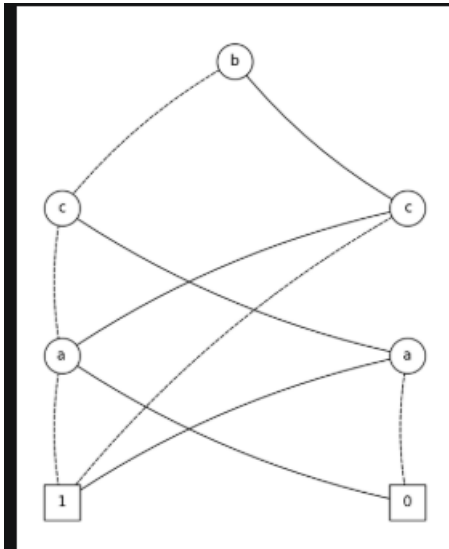


Figure 5: ROBDD generated for Test Case 5

Test Case 6

Input: $ABC + Abc + bC + aBc + ab$

Expression Number: 6

Number of Variables: 3

Variable Ordering: b, a, c

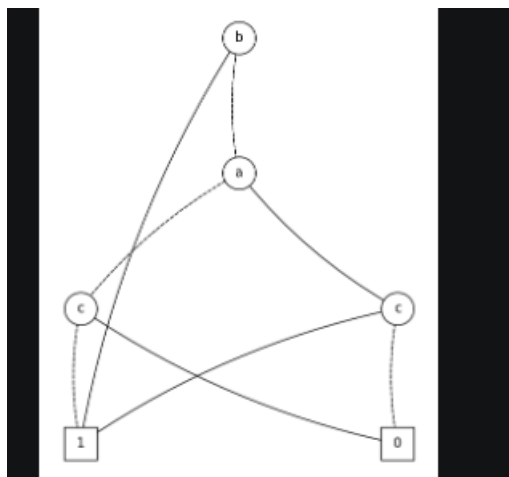


Figure 6: ROBDD generated for Test Case 6

Test Case 7

Input: $a + bc + DE + FGH + Af + IjK + lMN + bn + dhk + eln$

Expression Number: 7

Number of Variables: 14

Variable Ordering: n, a, b, d, e, f, h, k, l, c, g, i, j, m

Beginning of DFS Output

id=67 var=n low=46 high=66

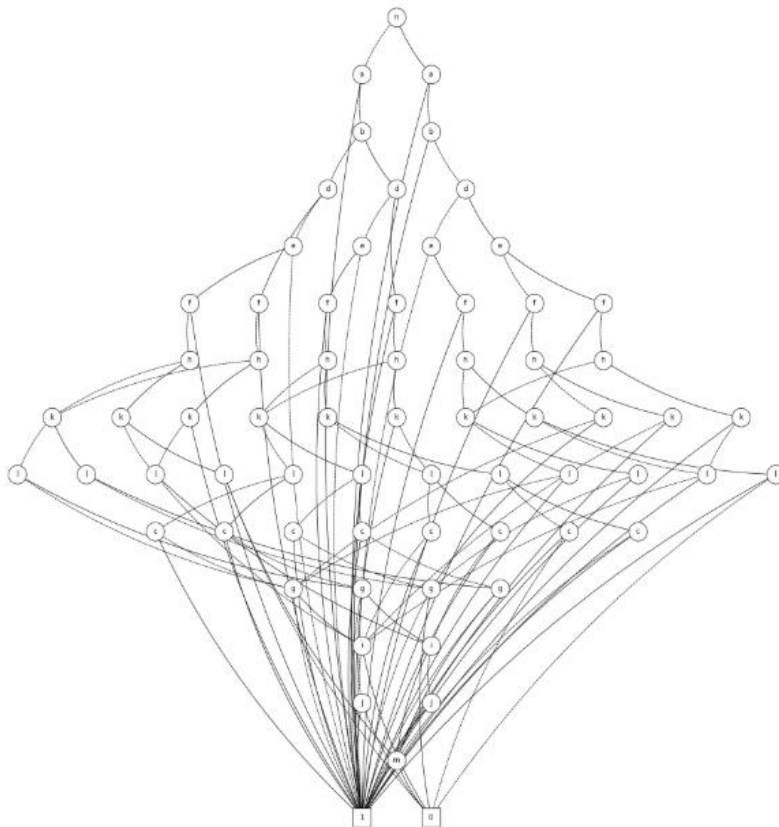


Figure 7: ROBDD generated for Test Case 7

Test Case Summary

Test Case	Expression	Number Variables	of Variable Ordering	Assignment Requirement
1	$a + Ab$	2	a, b	Does not satisfy minimum

Test Case	Expression	Number Variables	of Variable Ordering	Assignment Requirement
2	$aB + Ab$	2	a, b	Does not satisfy minimum
3	$ab + Cd + eF$	6	a, b, c, d, e, f	Satisfies
4	$ab + Cd + eF + Ag + BH$	8	a, b, c, d, e, f, g, h	Satisfies
5	$ABC + Abc + bC + aBc$	3	b, c, a	Does not satisfy minimum
6	$ABC + Abc + bC + aBc + ab$	3	b, a, c	Does not satisfy minimum
7	$a + bc + DE + FGH + Af + IjK + lMN + bn + dhk + eln$	14	n, a, b, d, e, f, h, k, l, c, g, i, j, m	Satisfies

Methodology

1. Reading the Boolean Expression

The main program opens the input text file and selects the expression corresponding to the expression number entered by the user.

The expression is then split using the + operator.

For example: $ABC + Abc + bC + aBc$ becomes: [ABC, Abc, bC, aBc]

2. Preprocessing the Expression

The product terms are passed through the `in_order()` function.

This function ensures that variables within each product term follow a consistent alphabetical order while preserving whether each variable is complemented or non-complemented.

3. Conversion to PCN

The Boolean expression is converted into the PCN representation using the `list_to_pcn()` function.

The program first identifies all variables present in the expression.

Each cube is then represented using: 1 → variable present 2 → complemented variable
3 → variable absent

This representation is used during recursive cofactor calculation.

4. Cofactor Calculation

The ROBDD is constructed using Shannon decomposition. $F = xFx + \bar{x}\bar{F}\bar{x}$

The program recursively calculates these two cofactors.

LOW (dashed line) → $x = 0$ HIGH (solid line) → $x = 1$

5. Merging Equivalent Nodes

The program also checks whether a node having the same variable and the same LOW and HIGH children already exists. The unique table UT is used for this purpose. If an equivalent node already exists, the existing node is reused instead of creating another node. This allows identical subgraphs to be shared. As a result, the final diagram is reduced and contains no duplicate equivalent nodes.

6. Computed Table

A computed table CT is also maintained. It stores previously processed Boolean subproblems so that the same subproblem does not need to be recursively evaluated again. This avoids unnecessary repeated computation during ROBDD construction. The combination of: recursive cofactor decomposition, redundant-node elimination, equivalent-node merging, computed-table lookup produces the final ROBDD.

ROBDD Node Representation

Each node is represented using the TreeNode class.

Each node contains: Node ID, Variable/literal, LOW child, HIGH child

Two terminal nodes are created at the beginning:

ID = 0 → Terminal 0 ID = 1 → Terminal 1

All non-terminal nodes are assigned IDs starting from 2.

The node representation therefore allows the ROBDD to be traversed and displayed.

DFS Traversal

The program performs a Depth First Search of the ROBDD. For every node, the following information is displayed: id = node ID, var = variable

low = LOW child ID high = HIGH child ID

For terminal nodes: id = node ID TERMINAL 0 or id = node ID TERMINAL 1

DFS is implemented recursively in the TreeNode class. A set of visited node IDs is maintained so that shared nodes are not visited multiple times.

BFS Traversal

The program also performs a Breadth First Search. A queue is used to visit nodes level by level. As with DFS, a set of visited node IDs is maintained to avoid processing shared nodes more than once. The BFS output provides another way of verifying the structure of the generated ROBDD.

Interpretation of the ROBDD

The edges are differentiated as: Dashed edge → LOW branch (0) and Solid edge → HIGH branch (1) The variable levels are arranged according to the variable ordering used during ROBDD construction.

Variable Nodes: Each circular node represents a Boolean variable.

Terminal Nodes: The two square terminal nodes represent the Boolean constants: 0 1

A path from the root to terminal 1 represents an input combination for which the Boolean function evaluates to 1, similarly for 0.

Reduction of the BDD

The ROBDD differs from an ordinary Binary Decision Diagram because redundant and equivalent nodes are removed.

Reduction 1 – Elimination of Redundant Nodes

If both branches of a node lead to the same child, then the node does not affect the function and is removed.

Reduction 2 – Merging Equivalent Nodes

If two nodes have: the same variable, the same LOW child, the same HIGH child, then they represent the same Boolean subfunction. Only one copy is retained and both references point to the same node.

Conclusion

The program successfully constructs a Reduced Ordered Binary Decision Diagram for Boolean expressions represented in Sum of Products form.

The Boolean expression is first divided into product terms and converted into a PCN representation. A variable ordering is then generated based on the occurrence frequency of variables. The ROBDD is constructed recursively by calculating the cofactors of the Boolean function with respect to the variables in the selected order.

The final ROBDD can be inspected using both DFS and BFS traversals and can also be visualized graphically using NetworkX and Matplotlib.

Resources

1. <https://pyeda.readthedocs.io/en/latest/bdd.html>
2. <https://github.com/cjdrake/pyeda/blob/main/pyeda/boolalg/bdd.py>
3. <https://github.com/abhinuypitale/ROBDD/tree/master>